ROMHACK



Command-Line Alchemy: Turning Scripts into Superpower

possible.lv

Kirils Solovjovs

```
cat access_log | grep 200.199.198.197
grep 200.199.198.197 access_log
grep -F 200.199.198.197 access_log
```

[A] bash -c "exit 0"

[B] bash -c "exit 1"

[C] false

[D] true



$POSIXLY_CORRECT

not posix compatible

[-z $POSIXLY_CORRECT]

```
#!/bin/bash
[ "${1:0:1}" = "-" ] && LINES="${1#-}" &&
shift
[ -z "$LINES" ] && LINES=10

LINEST=$(( $LINES / 2))
LINESH=$(( $LINES - $LINEST ))

[ $LINESH == 0 ] && exit
[ $LINEST == 0 ] && head -n$LINESH
"${1:--}" && exit
```

```
awk -v h=$LINESH -v t=$LINEST '
NR<=h{print;next}
{buf[NR%t]=$0}
END{
      if(NR<=h)exit
      if (NR>h+t)print "==> (" NR-h-t " more
lines) <=="
      cont=NR-t+1; if(cont<h+1)cont=h+1
      for(i=cont;i<=NR;i++)print buf[i%t]
}' "${1:--}"
```

- Write programs to handle text streams, because that is a universal interface.
- Write programs that do one thing and do it well.
- Write programs to work together.
- When a program has nothing surprising to say, it should say nothing.

Source: https://cscie2x.dce.harvard.edu/hw/ch01s06.html

```
cat access_log | grep -F 127.0.0.1 |  cut -d : -f 2|sort|uniq -c
```

I want to testrun this without grep and re-add it later. What's the best way to achieve that?

Wrong answers:

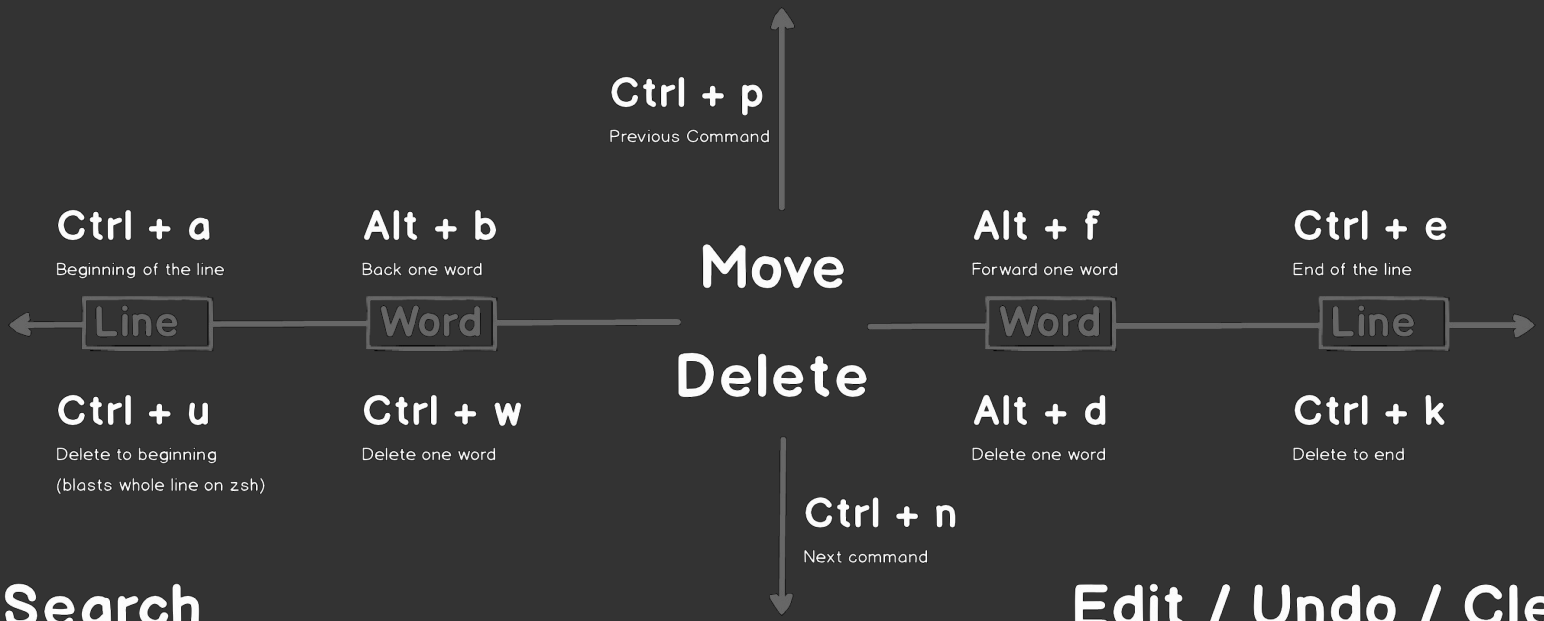
```
cat access_log # | grep -F 127.0.0.1 |  cut -d : -f 2|sort|uniq -c
```

```
cat access_log | : grep -F 127.0.0.1 | cut -d : -f 2|sort|uniq -c
```

cmdchallenge.com

Navigating the Command Line

github.com/dwmkerr/effective-shell



Search

Ctrl + r
Search Backwards

Ctrl + s
Search Forwards

Ctrl + g
Cancel

← , →
Edit

Edit / Undo / Clear

Ctrl + x,e
Edit

Ctrl + l
Clear

Cltr + -
Undo

Credit: effective-shell.com

- file descriptors -- stdin, stdout, stderr --, redirects
  - not redirects: << and <<<
- pipes, fifo, process subsitution, command substitution
  - | is not ||
- signals
- job control, subshells
  - & is not &&
- globbing, brace expansions
- variables: local, shell, env
- arithmetic

https://github.com/0ki/shellscripts/

```
cat <<EOF >script.sh
#!/bin/bash

while true; do
 echo ohi
 sleep 1.1
done
EOF
chmod a+x ./script.sh
```

```
cat <<EOF >script.sh
#!/bin/bash

while true; do
 echo ohi
 sleep 1.1
done
EOF
chmod a+x ./script.sh
```

```
$ ./script.sh
```

```
cat <<EOF >script.sh
#!/bin/bash

while true; do
 echo ohi
 sleep 1.1
done
EOF
chmod a+x ./script.sh
```

```
$ ./script.sh
ohi
ohi
ohi
ohi
ohi
ohi
```

```
cat <<EOF >script.sh
#!/bin/bash

while true; do
 echo ohi
 sleep 1.1
done
EOF
chmod a+x ./script.sh
```

```
$ timeout 3 ./script.sh
```

```
cat <<EOF >script.sh
#!/bin/bash

while true; do
 echo ohi
 sleep 1.1
done
EOF
chmod a+x ./script.sh
```

```
$ timeout 3 ./script.sh
ohi
ohi
ohi
```

```
cat <<EOF >script.sh
#!/bin/bash

while true; do
 echo ohi
 sleep 1.1
done
EOF
chmod a+x ./script.sh
```

```
$ timeout 3 ./script.sh | wc -l
```

```
cat <<EOF >script.sh
#!/bin/bash

while true; do
 echo ohi
 sleep 1.1
done
EOF
chmod a+x ./script.sh
```

```
$ timeout 3 ./script.sh | wc -l
Terminated
```

Why? Is it --
(a) a signal
(b) an exit code

```
$ timeout 3 ./script.sh | wc -l
Terminated
$ echo ${PIPESTATUS[@]}
124 143
```

```
$ (timeout 3 ./script.sh; echo )  | wc -l
4
```

```
$ (timeout 3 ./script.sh)  | wc -l
Terminated
```

```
$ cat ~/bin/unbuffer
#!/bin/sh
stdbuf -i0 -o0 -e0 "$@"
$ unbuffer timeout 3 ./script.sh  | wc -l
3
```

Why? Is it --
(a) a signal
(b) an exit code
(c) pipe buffers
(d) lack of EOF

```
$ cat ~/bin/unbuffer
#!/bin/sh
stdbuf -i0 -o0 -e0 "$@"
$ stdbuf -i0 -o0 -e0 timeout 3 ./script.sh  | wc -l
Terminated
```

```
$ /bin/sh -c "timeout 3 ./script.sh"  | wc -l
3
```



I think that's about enough of this

More funky
stuff here